

Ch#9

Cluster Analysis

Cluster:

⇒ collection of data objects that are similar to one another within the same cluster & different with other clusters.

what is cluster analysis?

Definition:

⇒ Cluster analysis (or clustering) is a process of grouping a set of data objects into subsets or clusters.

⇒ Objects in the same cluster are similar to each other and dissimilar to those in other clusters.

Requirements for cluster analysis

⇒ These are the following requirements for cluster analysis:

- Scalability - handle large dataset
- Handling different types of data.
- Detecting clusters of any shape.
- minimum domain knowledge required
- Robustness to noise.
- Capability to handle high dimensional sparse data.

- ⇒ Must consider specific constraints
- ⇒ interpretability and usability.
- ⇒ must update clusters with new data without starting over.
- ⇒ Insensitivity to i/p order.

Categories of clustering Techniques

Hierarchical methods

- Create a tree of clusters.

Partitioning method

- Divide data into a set number of clusters.

Density-based method

- Form clusters based on data density.

Grid-based method

- Use a grid structure to cluster data.

Types of clustering

1. Hierarchical methods

Def:

This method builds a tree-like structure of clusters without needing to know the number of clusters in advance.

Input:

A set of samples X and a similarity measure s .

Output:

A hierarchy of clusters.

Types of Hierarchical clustering

Agglomerative
Clustering

Divisible
Clustering

1. Agglomerative methods:

- o Bottom up approach
- o Start with individual objects as clusters.
- o Iteratively merge clusters to form large ones.

Process:

Initial steps:

- o Each object is a separate cluster.

Merging step:

- o Finds the two closest clusters based on a similarity measure.

Combining:

- o Merge these clusters into one.

Iterations:

- o Continues merging clusters until one large cluster remains or a stopping condition is reached.

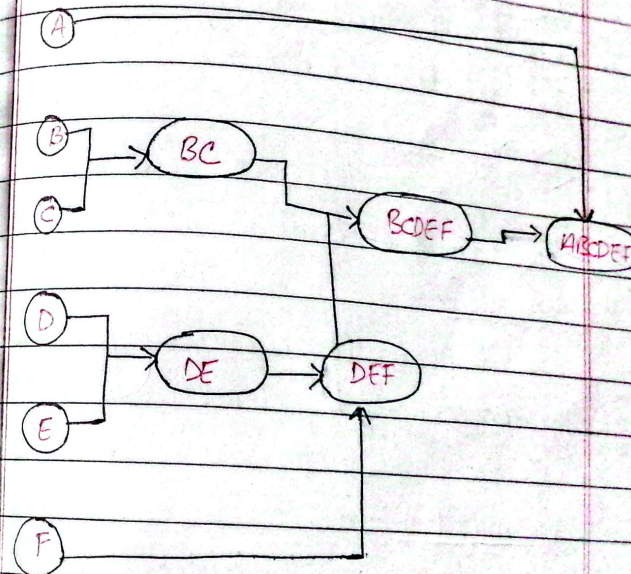
Maximum iterations:

- o Requires at most n iterations, where n is the number of objects.

Result:

- o A tree like diagram (dendrogram) showing how clusters were merged.
- o Cut the dendrogram at different levels to form different clusters.

Ex:



Step 1 Step 2 Step 3 Step 4 Step 5

Divisible Hierarchical method: (DIANA)

- o Top-down approach
- o Start with all objects in one cluster.
- o Iteratively split the cluster into smaller one.

Process:

Initial step: All objects are in one large cluster (the root).

splitting step:

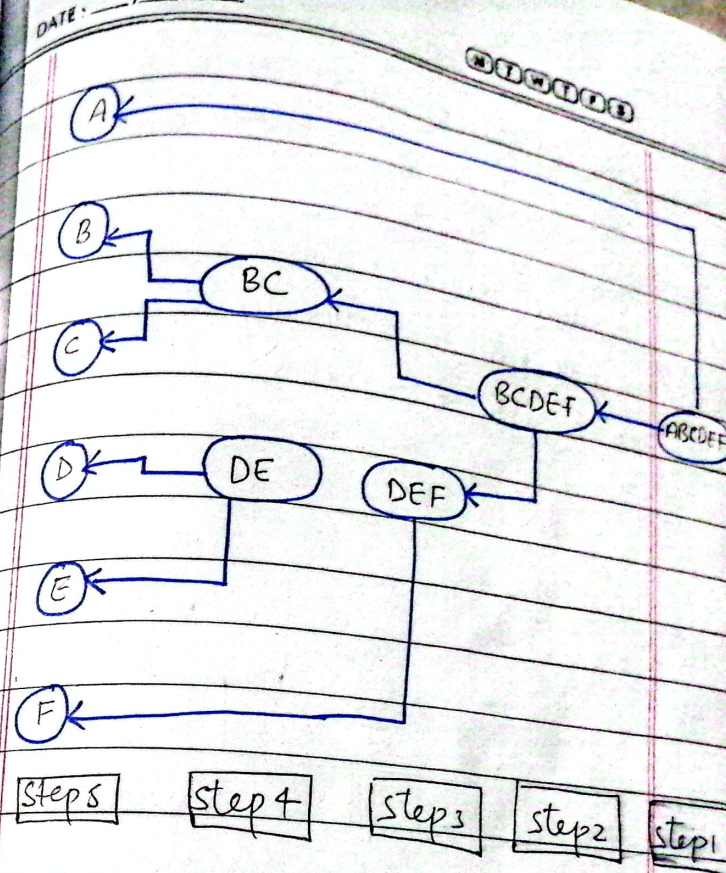
Divides the root cluster into smaller clusters.

Reursion:

Repeats the partitioning process on each resulting cluster.

Termination:

Stops when clusters are small enough or objects within clusters are similar.

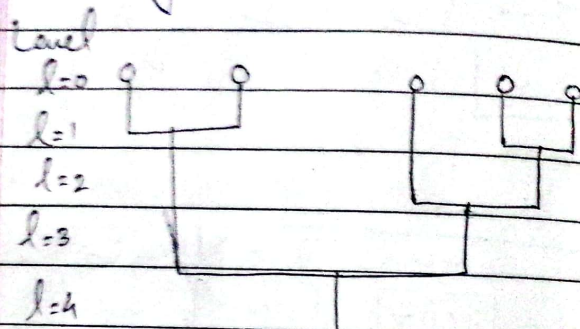


Pros:

- o Data Summarization
- o Data visualization
- o Helps understanding the data structure

Dendrogram representation of hierarchical clustering

A tree structure called a dendrogram is commonly used to represent the process of hierarchical clustering.



Single-link and Complete-link algorithms: (ways of calculating distance b/w two clusters)

Single link

- The distance between two clusters is the smallest distance between any two samples in the two clusters.

Complete-link

- The distance between two clusters is the largest distance between any two samples in the two clusters.

Chameleon Algorithm

- Chameleon is a hierarchical clustering algorithm that dynamically determines cluster similarity. → using multiple hierarchies clustering
- it evaluates interconnectivity within clusters and proximity b/w clusters to decide on merging.

Adaptability

- Chameleon adapts to cluster characteristics without relying on static user-defined models.
- it automatically adjusts its merging process based on internal cluster features.

Applicability:

- Suitable for all datatypes.

Algorithm

1. Graph construction:

- o Create a graph where each point is a node.
- o Connect nodes if one is among the k -nearest neighbours of the other.
- o Edges are weighted by how close the points are.

2. Graph partitioning:

- o Applies a graph partitioning algorithm to split the graph into small subclusters.
- o minimize edge cuts to ensure cohesive subclusters.

3. Iterative merging:

- o Employs an agglomerative hierarchical clustering algorithm to merge subclusters.
- Two clusters are merged if $\uparrow$ interconnectivity & closeness $\uparrow$ to internal.

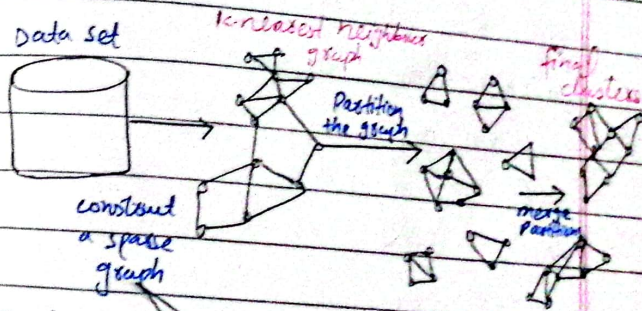
Cluster similarity metrics:

Relative interconnectivity:

- measures the absolute interconnectivity normalized w.r.t internal interconnectivity.

Relative closeness:

- Evaluate the absolute closeness normalized w.r.t internal closeness.



Evaluation:

- o Chameleon outperforms other algorithms like BIRCH and DBSCAN in discovering complex, high-quality clusters.

Processing cost:

- o Higher processing cost.

BIRCH: Algorithm

Stands for: Balanced iterative reduction and clustering using hierarchies

Definition:

Scalable

clustering method

it is

for data in Euclidean

spaces.

to

minibatch

kmeans.

in

1996

developed

by

Zhang,

Tian,

Raghu,

Ramakrishnan

& Livny

Efficient clustering technique for data in Euclidean vector spaces.

Designed for clustering large numeric datasets by integrating hierarchical clustering and iterative partitioning.

The algorithm efficiently clusters data with a single pass and deal effectively with outliers.

Fundamental Concepts:

BIRCH relies on the concept of CF (clustering features) and a CF tree for clustering.

CFs provides a compact representation of clusters.

Cluster of data points is represented by a triple number (N, LS, SS) where

N:

Number of items in subcluster / no of samples

LS:

Linear sum of the data points

SS:

Sum of the squared of the data points.

CFs offer a level of abstraction, simplifying the consideration of samples that are close enough to be considered as group.

CF-tree structure

CF-tree organize clustering features in a tree-structure, where non-leaf nodes aggregate information from their child nodes.

A CF tree has two parameters:

Parameters like branching factor and threshold regulate the tree's size and depth, influence impacting its efficiency and granularity.

branching factor: specifies the maximum number of children per non leaf node.

Threshold: specifies the maximum diameter of the leaf nodes.

Calculating cluster statistics:

- Cluster statistics like centroid, radius and diameter can be derived from clustering features.
- These statistics show how tightly packed the cluster is around its center.

Cluster Summarizations

- CFs allow for deriving various cluster statistics and distance measures.

Merging clusters

- Merging clusters based on their CF descriptions is straightforward and efficient.
- The CF entries of two disjoint clusters can be combined by simply adding their respective CF components.

DATE: 1/12

Incremental and dynamic clustering with CF trees

- BIOCH utilizes a hierarchical data structure called a CF tree for incremental and dynamic clustering.

- CF tree nodes maintain CF entries and are organized to efficiently partition incoming data points.

CF tree parameters

- CF trees have parameters like maximum branching factor (B) max. no. of children per non-leaf node and maximum entries (L) per leaf node for leaf nodes.
- a threshold parameter (T) limits the maximum diameter of a cluster entry → stored at the leaf node of the tree, influencing the tree size.

Building the CF tree

- The CF tree is constructed incrementally as data samples are scanned.

- Each level of the tree incorporates new data samples into the closest node, updating CF statistics along the path to the leaf node.

- Splitting can occur if a leaf node becomes over crowded, ensuring efficient utilization of memory.

Phases of Birch algorithm:

⇒ BIRCH involves multiple phases during the clustering process:

Phase 1: Scan all data and build an initial CF tree in memory:

- it linearly scans all samples and inserts them in CF tree described earlier.

Phase 02:

- Condense the tree to a desirable size by building a smaller CF tree.

DATE: 1-12-20

This can involve removing the outliers and further merging of clusters.

Phase-03:

Global clustering

- Employ a global clustering algorithm using the CF tree leaves as inputs. The fact that CF is balanced allows the log-efficient search.

Phase-4: Cluster refining

- This is optional, it re-scans more passes over data to refine the results.

Performance and extensions:

- ⇒ BIRCH typically achieves fast clustering with a single pass over the data and can improve the quality with additional passes.
- ⇒ offers extension to handle non-metric data.

DATE: 1/1/20

MTWTFSS

Advantages:

- ⇒ Demonstrate linear scalability
- ⇒ Produces high quality clusters
- ⇒ Efficient utilization of memory

Limitations:

- ⇒ CF tree nodes may not always correspond to natural clusters due to size limitations.
- ⇒ Performance may degrade for non-spherical clusters as BIRCH relies on radius or diameter to define cluster boundaries.

Applications beyond BIRCH:

- Clustering streaming
- Dynamic data

Derivations:

$$CF = CF_1 + CF_2$$

$$= (N_1 + N_2 + LS_1 + LS_2 + SS_1 + SS_2)$$

$$CF = \langle n, LS, SS \rangle$$

Ex:

$$C_1 = \{9, 12, 10, 8, 11\} \quad CF(S_1, S_2, S_3) \text{ where } n=5$$

$$LS = 9 + 12 + 10 + 8 + 11 = 50$$

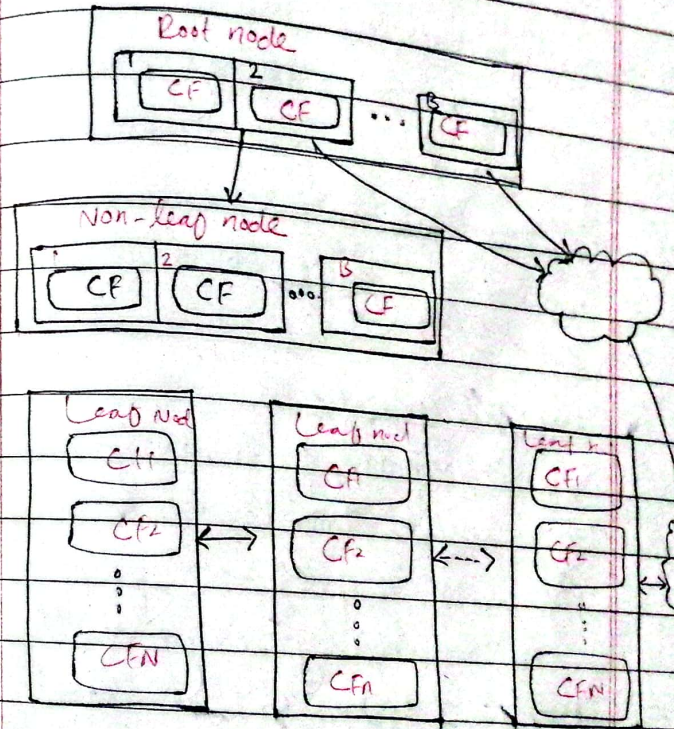
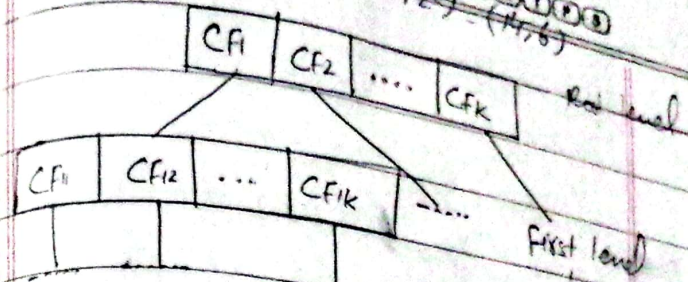
$$SS = 9^2 + 12^2 + 10^2 + 8^2 + 11^2 = 510$$

for 2D: $C_2 = \{(1,1), (2,1), (3,2)\}$

$$CF(C_2) = (3, (8,4), (14,6))$$

$$LS = (1+2+3, 1+1+2) = (6,4)$$

$$SS = (1^2+2^2+3^2, 1^2+2^2+2^2) = (14,6)$$



Terminology:

$$\text{Centroid: } x_0 = \frac{1}{n} \sum_{i=1}^n x_i$$

$$\text{Radius: } R = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - x_0)^2} \Rightarrow \frac{SS - (LS)^2/n}{N/N}$$

$$\text{Diameter: } D = \sqrt{\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^n (x_i - x_j)^2 / (n-1)}$$

2. Partitioning method:

DATE: 1/1/20

can be horizontal or vertical.

Definition: → Also known as centroid-based method.

Techniques used in data clustering where a dataset is divided into different groups or "partitions".

Initial partitioning:

- o The method starts by creating an initial set of k partitions.

The parameter k represents the number of groups to divide data into.

Iterative relocation:

- o After initial groups are formed, the method tries to improve the quality of these groups. it does this by moving objects from one group to another.

⇒ Partitioning clustering algorithms create a single partition of data, unlike hierarchical

DATE: 1/1/20

techniques that produce a clustering structure, such as dendrogram.

Global and local criteria:

Global criterion:

- o Optimizes a function defined over all samples. for example, the Euclidean square error measure represents each cluster by a centroid and assigns samples to cluster based on similarity to these centroids.

Local criterion:

- o Forms clusters using the local structure in the data.

Square-error criterion:

⇒ The most commonly used partitional clustering strategy is based on the square error criterion, aiming to minimize the total square-error for fixed number of clusters.

Mean vector

The centroid of a cluster.

$$M_k = \frac{1}{n_k} \sum_{i=1}^{n_k} x_{ik}$$

where x_{ik} is the i th sample in cluster C_i .

within-cluster variations

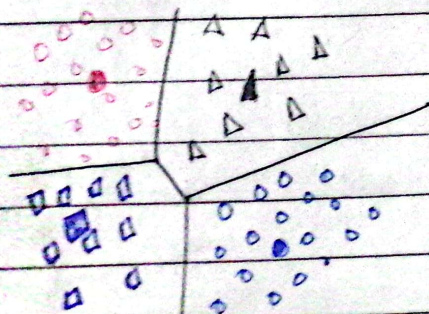
- Sum of squared Euclidean distances b/w each sample x_k and its centroid.

$$e_k^2 = \sum_{i=1}^{n_k} (x_{ik} - M_k)^2$$

Total-square error

- Sum of within-cluster variations for all clusters.

$$E^2 = \sum_{k=1}^K e_k^2$$



K-Mean Algorithm

Def:

It is an iterative algorithm that divides the unlabeled data set into K different clusters in such a way that each dataset belongs only one group that has similar properties.

Centroid-based:

It is centroid-based algorithm

main aim:

The main aim of this algorithm is to minimize the sum of distances between the data point and their corresponding clusters.

Concepts

Dataset: D with n objects in Euclidean space.

Clusters: Divide D into K clusters $(C_1, \dots, C_K)$.

Centroid:

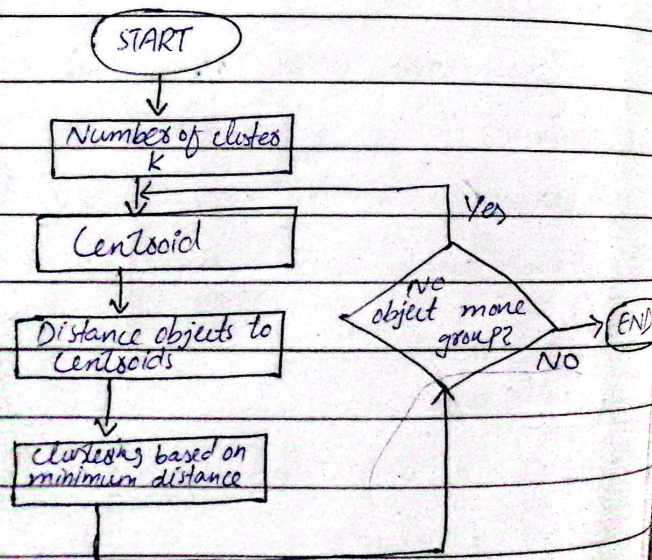
- Represent the center of the clusters, calculated as the mean of the objects in the clusters.
⇒ update each time when new data added to a cluster.

Objective function:

$$E = \sum_{i=1}^k \sum_{P \in C_i} \text{dist}(P, C_i)^2$$

Goal is to minimize within-cluster variation.

K-mean clustering flowchart:



DATE: 1/20

Steps of k-means

1. Initial position:

Randomly chooses k samples as initial cluster centroids.

2. Assign samples:

Assign each sample to the closest cluster centroid.

3. Recompute centroids:

Calculate new centroids for each cluster based on the current cluster membership.

4. Repeat:

Repeat 2 and 3 until no sample reassignment reduces the total square-error, indicating convergence.

Advantages of k-means:

Time Complexity:

Linear time complexity

$$O(n \times k \times l)$$

clusters sample iterations

MTWTFES

These are two main steps:
1. Data assigned
2. centroid update

Space Complexity:

⇒ $O(k+n)$

Order independence:

⇒ Algorithm produces the same partition irrespective of sample presentation order.

Challenging with k-means:

1. Choosing the number of clusters k:

No clear guidelines for determining optimal number of clusters.

2. Initial partition selection:

Sensitive to initial partition, which may lead to convergence at a local minimum.

3. Noise and outliers:

Highly sensitive to noise & outliers and also affect the mean value.

⇒ Difficult to discover clusters with non-

Data type limitation:

Not suitable for nominal data.

Pseudocode:

The k-means algorithm for partitioning, where each cluster center is represented by the mean value of the object in the center.

i/p:

- k : is the number of clusters
- D : Data containing n objects.

o/p:

- A set of k clusters

method:

- (1) Arbitrary choose k objects from D as the initial cluster centers.
- (2) repeat
- (3) (a) assign each object to the cluster to which object is the most similar, based on the mean value object in the cluster.
- (4) update the cluster mean
- (5) Until no change.

K-medoids method:

- ⇒ The K-medoids improves robustness by using the most centrally located objects in a cluster as the representative, making it less sensitive to noise and outliers compared to K-means.
- ⇒ Complexity $O(k(n-k)^2)$.

Computational challenge:

NP-hard problem:

Difficult to find exact solution for large k or high dimensional space.

Greedy approach:

Not necessarily find optimal solutions.

Fuzzy C-means:

- ⇒ An extension of K-means where each data point can belong to multiple clusters with a membership value expressed through fuzzy sets.

DATE: 1/1/20

Example of K-means:

Dataset:

Consider the following five data points:

$$x_1 = (1.0, 1.0)$$

$$x_2 = (1.5, 2.0)$$

$$x_3 = (3.0, 4.0)$$

$$x_4 = (5.0, 7.0)$$

$$x_5 = (3.5, 5.0)$$

Partition these data sets into 2 clusters $k=2$.

1. Initialize centroids:

Randomly select 2 initial centroids.

$$m_1 = (1.0, 1.0) \text{ (for } c_1)$$

$$m_2 = (5.0, 7.0) \text{ (for } c_2)$$

2. Assign points to the closest centroid:

Calculate Euclidean distance

$$d(x_1, m_1) = \sqrt{(1.0-1.0)^2 + (1.0-1.0)^2} = 0$$

$$d(x_1, m_2) = \sqrt{(1.0-5.0)^2 + (1.0-7.0)^2} \approx 7.21$$

$$d(x_1, m_1) = \sqrt{(1.5 - 1.0)^2 + (2.0 - 1.0)^2} \approx 1.41$$

$$d(x_1, m_2) = \sqrt{(1.5 - 5.0)^2 + (2.0 - 1.0)^2} \approx 6.10$$

$$d(x_3, m_1) = \sqrt{(3.0 - 1.0)^2 + (4.0 - 1.0)^2} \approx 3.61$$

$$d(x_3, m_2) = \sqrt{(3.0 - 5.0)^2 + (4.0 - 7.0)^2} \approx 3.61$$

$$d(x_4, m_1) = \sqrt{(5.0 - 1.0)^2 + (7.0 - 1.0)^2} \approx 7.21$$

$$d(x_4, m_2) = \sqrt{(5.0 - 5.0)^2 + (7.0 - 7.0)^2} = 0$$

$$d(x_5, m_1) = \sqrt{(3.5 - 1.0)^2 + (5.0 - 1.0)^2} \approx 4.71$$

$$d(x_5, m_2) = \sqrt{(3.5 - 5.0)^2 + (5.0 - 7.0)^2} \approx 2.50$$

Initial cluster assignment

$$C_1 = \{x_1, x_2\}$$

$$C_2 = \{x_3, x_4, x_5\}$$

Recompute centroids:

$$m_1 = \left(\frac{1.0 + 1.5}{2}, \frac{1.0 + 2.0}{2} \right) = (1.25, 1.5)$$

$$m_2 = \left(\frac{3.0 + 5.0 + 3.5}{3}, \frac{4.0 + 7.0 + 5.0}{3} \right) \approx (3.83, 5.33)$$

DATE: 1. 1.20

Repeat step 2 and 3:

Reassign points based on the new centroid and recompute the centroids until no further changes occur.

Since the clusters assignment have no changed, the algorithm has converged.

Final clusters:

$$C_1 = \{(1.0, 1.0), (1.5, 2.0)\}$$

$$C_2 = \{(3.0, 4.0), (5.0, 7.0), (3.5, 5.0)\}$$

Final centroids:

$$m_1 = (1.25, 1.5)$$

$$m_2 = (3.83, 5.33)$$

- Simple and easy implement
- Quite efficient
- Often terminate at local optimum.

3. Density-Based method:

1996 by Martin Ester, Hans-Peter Kriegel, Jörg Sander & Xiaojin Xie

⇒ Density-based clustering is a method that identifies clusters in data based on the density of data points in a region.

⇒ Unlike partitioning and hierarchical methods which often find spherical clusters, density-based methods can identify clusters of arbitrary shapes.

DBSCAN Algorithm

Density-Based spatial clustering of Applications with Noise.

Definition:

⇒ Density-based clustering algorithm used for identifying clusters of arbitrary length/shape in a dataset. It assumes that clusters are dense regions separated by regions of low density (noise).

Key Concepts

Density:

Clusters exhibit a higher density of points compared to regions of noise. This characteristic helps DBSCAN distinguish between clusters and noise. The number of points within

Connectivity:

a neighbourhood. DBSCAN considers the local distribution of nearest neighbours to determine the density and connectivity of points within a cluster.

MinPts: (minimum no. of points in a circle or cluster)

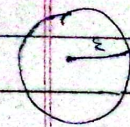
• The minimum number of points required in a neighbourhood for it to be considered dense.

Neighbourhood:

• The area around a point.

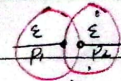
Radius (ϵ):

• The distance from the point within which we consider other points to be in the neighbourhood.



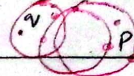
Density Reachability

⇒ A point p_1 is density reachable from another point p_2 if p_1 is within the epsilon neighbourhood of p_2 and the density around p_2 exceeds a minimum threshold (m) .



Density Connectivity

⇒ Points p_0 and p_1 are density-connected if there exists a sequence of density-reachable points between them.



Classification of points

Core points:

⇒ A point is core point if its neighbourhood contains at least $minPts$.

⇒ Points with a sufficient number of neighbours within their epsilon neighbourhood. They form interior of clusters.

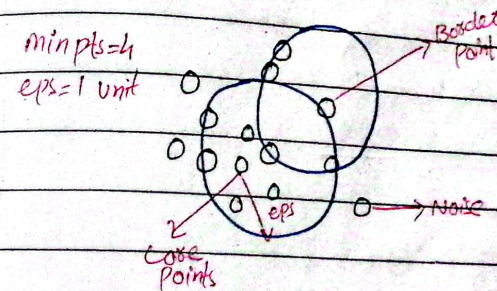
DATE: 1/120

Border point:

⇒ Points with fewer neighbours than core points but are within the epsilon neighbourhood of a core point. They lie on the edge of the clusters.

Noise points:

⇒ Points that are neither core nor border points. They typically represent outliers.



Process:

Point selection:

- Arbitrary select a point.

Density reachability:

- Retrieve all points density-reachable from the

selected point based on ϵ and m .

3. Cluster formation:

- if the selected point is core point, form a new cluster or extend an existing one.
- if it's border, move to the next point.

4. Continue:

- Repeat the process for all points until all points have been processed.

5. Cluster merging:

- Clusters may be merged if they are close to each other i.e. distance b/w clusters $\leq \epsilon$.

Pseudocode:

I/P:

D : A dataset containing n objects

ϵ : The radius parameter.

MinPts : The neighbourhood density threshold.

output:

A set of density-based clusters.

Initialization:

mark all objects as unvisited.

Cluster formation loop:

Repeat until all objects visited.

1. Randomly select an unvisited object P .

2. mark P as visited.

3. if the ϵ -neighbourhood of P has at least MinPts object:

- Create a new cluster C , and add P to C .

- Let N be the set of objects in the ϵ -neighbourhood of P .

of P .

$\Rightarrow$ if P' is unvisited.

- mark P' as visited.

- if the ϵ -neighbourhood of P' has at least MinPts ,

add those points to N .

4. if P' is not a member of any cluster add P' to C .

Advantages:

- Not require prior knowledge to form number of clusters.
- Can find clusters of an arbitrary shape.
- Can find noise point and handles outliers.
- Parameters sensitivity: requires only two parameters ϵ and m

Disadvantages:

Complexity

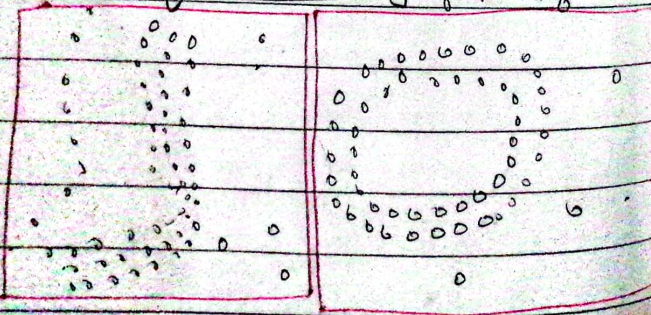
Complexity is very high.

$$O(n \times \log n)$$

Curse of dimensionality:

may encounter issue with high dimensional datasets.

Clusters of an arbitrary shape:



DATE: 1/20

Conclusion:

DBSCAN is effective for low-dimensional datasets and offers advantages in handling arbitrary shaped clusters and noise points. However, it may require careful parameter tuning and optimization for high-dimensional datasets.

Example:

$$D = \{ (1,1), (2,2), (2,3), (8,8), (8,9), (25,80) \}$$

Use the following parameters:

- $\epsilon = 2$, radius parameter
- $\text{MinPts} = 2$, minimum number of points required to form a dense region.

Initialization:

- mark all points as unvisited.

Select unvisited point:

- Start with the first unvisited point (1,1)

mark as visited:

- Mark $(1,1)$ as visited.

Close point check:

- Check the ϵ -neighbourhood of $(1,1)$. it includes the points within a distance of '2' from $(1,1)$.
The points of the ϵ -neighbourhood of $(1,1)$ are $\{(1,1), (2,2)\}$
form a cluster:

- Since the ϵ -neighbourhood contains at least MinPts (2 in case) create a new cluster ' c_1 ' & add $(1,1)$ and $(2,2)$ to c_1 .
Expand the cluster:

- for each point in the neighbourhood of $(1,1)$, check their neighbourhoods.

for $(2,2)$, the ϵ -neighbourhood is $\{(1,1), (2,2), (2,3)\}$ add $(2,3)$ to c_1 .

Now, c_1 is $\{(1,1), (2,2), (2,3)\}$

DATE: 1/120

MTWTFSS

Select next unvisited point:

~~mark~~ select next unvisited point $(8,8)$.

mark as visited:

mark $(8,8)$ as visited.

Close point check:

- Check the ϵ -neighbourhood of $(8,8)$.

The points in the neighbourhood are $(8,9)$

form a cluster:

- Create a new cluster c_2 add $(8,8)$ and $(8,9)$ to ' c_2 '

Expand the cluster:

$\Rightarrow$ for each point in the neighbourhood of $(8,8)$, check their neighbours.

$\Rightarrow$ for point $(8,9)$, the ϵ -neighbourhood is $\{(8,8), (8,9)\}$.

Since both points are already visited and c_2 no point is added. $c_2 = \{(8,8), (8,9)\}$

Principles of incremental clustering

The most popular approach follows step:

1. Initial assignment:

⇒ The first data item is assigned to the first cluster.

2. Sequential assignment:

⇒ Each subsequent data item is assigned to an existing cluster or new one based on some criterion.

Iteration:

⇒ Repeat until all samples are clustered.

Advantages:

- Space efficiency
- Time efficiency

Weakness:

- Order sensitive.

5. Grid based method:

It takes different approach compared to data-driven approach.

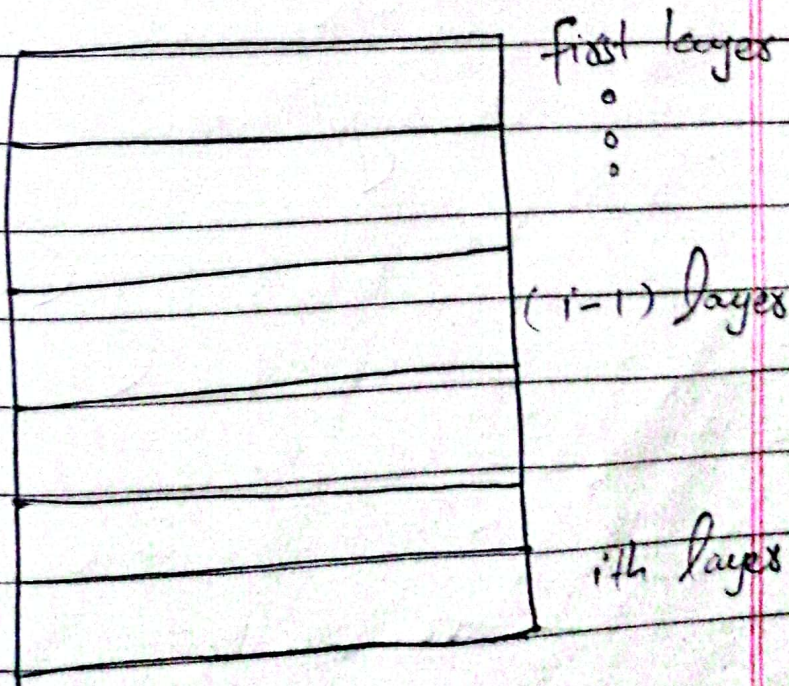
Grid based clustering partitions the entire embedding space into a finite number of cells, forming a grid structure.

Characteristics:

- Space driven partitioning
- fast processing time
- multi-resolution.

Example:

- Statistical information Grid (STING)
- CLIQUE



Applications of clustering:

- Business intelligence
- image pattern recognition
- web search
- Biology
- Security

Q. Role of clustering in data mining?

⇒ Clustering is a dynamic field of research in data mining. It is related to unsupervised learning in machine learning.

⇒ Cluster analysis can be used as a standalone data mining tool to gain insight into data distribution, or as a preprocessing step for other data mining algorithms operating on the detected clusters.

⇒ Clustering is a challenging field. Typical requirements: domain knowledge, scalability etc.

Advantages:

- ⇒ Pattern discovery
- ⇒ Data reduction
- ⇒ Improved understanding
- ⇒ Data preprocessing
- ⇒ Versatility
- ⇒ Anomaly detection

Disadvantages:

- ⇒ Parameters sensitivity
- ⇒ Scalability issues
- ⇒ Shape assumptions
- ⇒ Hard to interpret
- ⇒ Initialization dependency